

Problem 9 – Smart Quiz Master

Difficulty: ★★ Beginner

Overview

Create a Streamlit Quiz Application that allows students to answer multiple-choice questions, receive instant feedback, and view their final results.

The application should display one question at a time, keep track of the student's score, and present a clean, user-friendly interface.

Project Requirements

1. Store the Questions

Store the quiz questions in a list of dictionaries.

Each dictionary should contain:

- Question
- Option A
- Option B
- Option C
- Option D
- Correct Answer

Create at least 5 questions.

Example:

Question	Correct Answer
What is the capital of Egypt?	Cairo
$5 \times 8 = ?$	40
Python is a...?	Programming Language

2. Student Information

Before starting the quiz, ask the student to enter:

- Student Name

The quiz should not begin until a valid name is entered.

3. Quiz Flow

Display one question at a time.

For every question:

- Show the question.
 - Display the four answer choices.
 - Allow the student to select one answer.
 - Provide a Submit Answer button.
 - After submitting, display whether the answer is correct or incorrect.
 - Allow the student to continue using a Next Question button.
-

4. Calculate the Score

Award:

- 1 point for every correct answer.

At the end of the quiz calculate:

- Total Questions
 - Correct Answers
 - Wrong Answers
 - Final Score
 - Percentage
-

5. Display the Final Report

When the quiz is finished, display a summary similar to:

Quiz Report

Student: Karim

Correct Answers: 4

Wrong Answers: 1

Final Score: 4 / 5

Percentage: 80%

Excellent Work!

Bonus

After each submitted answer:

- Display Correct  or Incorrect .
 - If the answer is incorrect, show the correct answer before moving to the next question.
-

Streamlit Requirements

Required Text Functions

Use all of the following:

- `st.title()`
 - `st.header()`
 - `st.subheader()`
 - `st.write()`
 - `st.divider()`
-

Required Input Widgets

Use:

- `st.text_input()` → Student Name
 - `st.radio()` or `st.selectbox()` → Answer choices
 - `st.button()` → Submit Answer
 - `st.button()` → Next Question
-

Required Status Elements

Use at least four of the following:

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Suggested uses:

- Quiz instructions
- Correct answer
- Incorrect answer
- Last question warning

Layout Requirements

Sidebar

Display:

- Quiz Rules
 - Total Questions
 - Passing Score
-

Tabs

Create three tabs.



Tab 1 – Quiz

This tab is where the student answers the quiz.

Use two columns.

Left Column

Display:

- Student Name
- Current Question
- Answer Choices
- Submit Answer button
- Next Question button

Right Column

Display:

- Current Score
 - Current Question Number
 - Remaining Questions
-

Tab 2 – Results

After the quiz is completed, display:

- Student Name
- Total Questions
- Correct Answers
- Wrong Answers
- Final Score
- Percentage
- Pass/Fail
- Performance Message

Example messages:

- 🌟 Excellent Work!
 - 👍 Good Job!
 - 📖 Keep Practicing!
-

Tab 3 – Review Answers (Bonus)

Allow the student to review every question.

For each question display:

- Question
 - Student's Answer
 - Correct Answer
 - Result ( Correct or  Incorrect)
-

Expected Interface

 Smart Quiz Master

Sidebar

Quiz Rules

Total Questions : 5

Passing Score : 60%

Tabs

[ Quiz] [ Results] [ Review Answers]

Student Name

[Karim]

Question 3 of 5

Python is a...

☐ Programming Language

☐ Operating System

☐ Database

☐ Web Browser

[Submit Answer]

Current Score : 2

Remaining Questions : 2

 Correct!

Challenge ★★

Add three difficulty levels.

The student chooses:

- Easy
- Medium
- Hard

Each difficulty should have its own set of questions.

At the end, also display:

- Difficulty Level
 - Final Score
 - Percentage
 - Pass / Fail
-

Suggested Functions

Organize your code into reusable functions.

Examples:

- load_questions()
 - submit_answer()
 - next_question()
 - calculate_score()
 - display_results()
-

Python Topics Covered

- Variables
 - Strings
 - Lists
 - Dictionaries
 - Arithmetic Operators
 - Comparison Operators
 - Assignment Operators
 - if, elif, else
 - Functions
-

Streamlit Topics Covered

Text

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.radio()` or `st.selectbox()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`
- `st.session_state`

Learning Objectives

After completing this project, you should be able to:

- Store quiz data using a list of dictionaries.
- Build an interactive Streamlit application.
- Manage application state with `st.session_state`.
- Display one question at a time.
- Calculate scores and percentages.
- Organize code into reusable functions.
- Create a clean interface using sidebars, tabs, and columns.